PurityIQ — Developer-Executable Data Infrastructure Reference

Prepared: June 7, 2026. Every source below is verified for commercial use (PurityIQ is a commercial iOS app). Where commercial use is restricted, it is flagged explicitly. All URLs are full `https://` . API endpoints include curl + Python/Node.js where applicable.

TL;DR

- Build on **DeepSeek V4 Flash via Fireworks AI** (US-hosted, OpenAI-compatible) for the LLM PDF pipeline; **Apple VisionKit `DataScannerViewController`** for barcode scanning; **Open Food Facts API** for food products + **California "Chemicals in Cosmetics" CSV** for cosmetics; **Firebase Functions v2 + Firestore** for orchestration.
- Most government sources are **manual/bulk download (CSV/Excel/PDF)**, not real-time APIs. The clean exceptions with true APIs: **eCFR, openFDA, EPA Envirofacts (SDWIS), UK FSA regulated-products, and California's CKAN portal.**
- Two sources are **NOT commercially clean — EWG Tap Water Database (prohibited)** and **IARC Monographs (likely WHO non-commercial license — verify first)**. **DeepSeek-OCR is open-weights/self-host only**, not on the managed API.

SECTION 1 — GOVERNMENT FOOD & PESTICIDE DATA SOURCES

1.1 USDA Pesticide Data Program (PDP)

- **Landing/download:** https://www.ams.usda.gov/datasets/pdp/pdpdata · **Searchable DB:** https://apps.ams.usda.gov/pdp
- **Format:** ZIP of delimited ASCII/text files (e.g., `Pdp24Results.txt` , `Pdp24Samples.txt`), Access `.MDB` , reference `.xls` , plus machine-readable `PDP_DataDictionary.csv` . Annual summary is PDF (`PDPAnnualSummary.pdf`).
- **API:** None — manual annual download only. Mirrors: Ag Data Commons (figshare) https://agdatacommons.nal.usda.gov/articles/dataset/Pesticide_Data_Program/24660462 and data.gov.
- **File URL pattern:** `https://www.ams.usda.gov/sites/default/files/media/{YEAR}PDPDatabase.zip` . Confirm exact

current-year filename on the landing page.

- **Auth:** None. **Commercial use:** Yes (US public domain). **Attribution:** Not required; cite USDA AMS PDP. **Cadence:** Annual (CY2023 data released 2024; CY2024 summary present).
- **Extract:** sample identity (commodity, origin, site), analytical results (pesticide, concentration, LOD), lab, testing procedure per sample.
- ⚠️ **Known issues:** Detection limits vary by pesticide/lab/commodity/year; commodity list changes annually; pesticide screen expanded over time; pre-1994 = positive detections only. Column/file names embed 2-digit year tokens (`##`) — parser must handle year-specific names.

1.2 EPA 40 CFR Part 180 (pesticide tolerances)

- **eCFR:** https://www.ecfr.gov/current/title-40/chapter-I/subchapter-E/part-180 · **Specific tolerances (Subpart C):** https://www.ecfr.gov/current/title-40/chapter-I/subchapter-E/part-180/subpart-C
- **Format/API:** via eCFR API (§1.3). **Auth:** open. **Commercial use:** Yes. **Cadence:** continuous (Title 40 amended through May 2026).
- **Extract:** tolerance levels (ppm) by pesticide × commodity from §180 Subpart C.

1.3 eCFR API

- **Base URL:** `https://www.ecfr.gov` · **Docs:** https://www.ecfr.gov/developers/documentation/api/v1 · https://www.ecfr.gov/reader-aids/ecfr-developer-resources
- **Auth:** None (no API key). **Key endpoints:**
  - Full content (XML): `GET https://www.ecfr.gov/api/versioner/v1/full/{YYYY-MM-DD}/title-40.xml?part=180`
  - Structure (JSON): `GET https://www.ecfr.gov/api/versioner/v1/structure/{YYYY-MM-DD}/title-40.json`
  - Titles metadata: `GET https://www.ecfr.gov/api/versioner/v1/titles.json`
- **curl:** `curl "https://www.ecfr.gov/api/versioner/v1/full/2026-05-13/title-40.xml?part=180" -o part180.xml`
- **Python:**

```
import requests
r = requests.get("https://www.ecfr.gov/api/versioner/v1/full/2026-05-13/title-40.xml",
                 params={"part": "180"}, timeout=120)
r.raise_for_status(); open("part180.xml","wb").write(r.content)  # parse with lxml
```

- **Node.js:**

```
import fs from "node:fs";
```

```
const res = await fetch("https://www.ecfr.gov/api/versioner/v1/full/2026-05-13/title-40.xml'
fs.writeFileSync("part180.xml", Buffer.from(await res.arrayBuffer()));
```

- ⚠️ Returns XML (not clean JSON); tolerance values are in regulatory prose tables — needs XML parsing + the LLM pipeline to normalize.

1.4 FDA Metals and Your Food

- **URL:** https://www.fda.gov/food/chemical-contaminants-metals-pesticides-food/metals-and-your-food · **Hubs:** https://www.fda.gov/food/environmental-contaminants-food/closer-zero-reducing-childhood-exposure-contaminants-foods · https://www.fda.gov/food/environmental-contaminants-food/environmental-contaminants-food
- **Format:** HTML narrative + linked PDF data tables (Total Diet Study, Combination Metals Testing); per-contaminant pages (lead/arsenic/mercury/cadmium) hold test data. No structured CSV/API.
- **Auth:** None. **Commercial use:** Yes. **Cadence:** irregular (latest testing results April 29, 2026, infant formula / Operation Stork Speed).
- **Extract:** FDA action levels + Total Diet Study element results. Per FDA's **January 6, 2025 Final Guidance, "Action Levels for Lead in Processed Food Intended for Babies and Young Children": 10 ppb** for fruits, vegetables, mixtures, yogurts, custards/puddings, single-ingredient meats; **20 ppb** for single-ingredient root vegetables and dry infant cereals. FDA estimates these levels reduce dietary lead exposure **19–29% at the 90th percentile**. ⚠️ Action levels are guidance, not binding limits; Safe Food Alliance require PDF extraction.

1.5 California AB 899 (baby food heavy metals)

- **There is NO centralized state data portal.** AB 899 (H&SC §§110962–110963) requires each **manufacturer** to publish test results (arsenic, cadmium, lead, mercury) on **its own website**, accessed via QR codes on labels.
- **CDPH FAQ:** https://www.cdph.ca.gov/Programs/CEH/DFDCS/Pages/FDBPrograms/FoodSafetyProgram/AB899FAQ.aspx
- **Format:** Decentralized HTML per manufacturer (e.g., gerber.com). No CSV/Excel/API. CDPH will **not** initiate rulemaking and does not aggregate data.
- **Auth:** None. **Commercial use:** public disclosures; per-site terms vary. **Cadence:** monthly testing; disclosure for products produced on/after Jan 1, 2025.
- **Extract:** per-batch toxic element levels + product identifiers (UPC, lot). ⚠️ **Major data-quality issue:** each manufacturer formats differently; some gate behind sell-by/UPC/CAPTCHA. No standard schema — per-vendor scraping required. (March 2026: CA AG Bonta issued an enforcement advisory; CDPH has also interpreted scope to include some dietary supplements.)

1.6 FDA GRAS Notices

- **Inventory:** https://www.fda.gov/food/generally-recognized-safe-gras/gras-notice-inventory · **Searchable DB:** https://www.hfpappexternal.fda.gov/scripts/fdcc/index.cfm?set=grasnotices
- **Format:** Searchable web table with **"Download data in Excel format"** export. No REST API.
- **Auth:** None. **Commercial use:** Yes. **Cadence:** ongoing. **Extract:** GRN number, substance, notifier, intended use, FDA response letter + date.

1.7 FDA CAERS

- **Page:** https://www.fda.gov/food/compliance-enforcement-food/cfsan-adverse-event-reporting-system-caers · **openFDA:** https://open.fda.gov/data/caers/ (endpoint `https://api.fda.gov/food/event.json`)
- **Format:** Bulk CSV (Product-Based + Case-Based) + JSON via openFDA. README first: https://www.fda.gov/files/food/published/Read-Me-File-For-CFSAN-Adverse-Event-Reporting-System-Quarterly-Data-Extract.pdf
- **Auth:** None (optional openFDA key raises rate limits). **Commercial use:** Yes. **Cadence:** Quarterly.
- **Extract:** report ID, product, MedDRA-coded symptoms, outcomes, demographics (foods/supplements/cosmetics). ⚠️ Report ≠ causation; data are as-reported.

1.8 FDA Import Alerts + Food Recalls

- **Food enforcement (recalls) openFDA API:** `https://api.fda.gov/food/enforcement.json` — docs https://open.fda.gov/apis/food/enforcement/ · **Bulk:** https://open.fda.gov/apis/food/enforcement/download/ (zipped JSON)
- **curl:** `curl "https://api.fda.gov/food/enforcement.json?search=distribution_pattern:nationwide&limit=5"`
- **Format:** JSON API + zipped JSON, 2004–present, updated weekly. **Auth:** None (optional key). **Commercial use:** Yes. **Extract:** recall reason, product, classification, distribution, firm.
- ⚠️ **Import Alerts specifically** (detention-without-physical-examination red/green/yellow lists) live at https://www.accessdata.fda.gov/cms_ia/ialist.html and are **HTML-only, no structured export/API** — scrape required. openFDA enforcement is the structured proxy for recall data.

SECTION 2 — TAP WATER DATA SOURCES

2.1 EPA SDWIS — via Envirofacts REST API

- The current programmatic access is **EPA Envirofacts RESTful API** (the legacy SDWIS/FED reporting app is form-based, not a clean API).
- **Docs:** https://www.epa.gov/enviro/envirofacts-data-service-api ·

https://www.epa.gov/enviro/download-additional-envirofacts-datasets

- **Base pattern:** `https://data.epa.gov/efservice/{TABLE}/{COLUMN}/{VALUE}/{FORMAT}`
- **By PWSID:** `https://data.epa.gov/efservice/SDWIS_WATER_SYSTEM/PWSID/CA1234567/JSON` · **By ZIP:** `https://data.epa.gov/efservice/SDWIS_GEOGRAPHIC_AREA/ZIP_CODE_SERVED/90001/JSON`
- **Formats:** JSON, CSV, Excel, XML. **Auth:** None.
- **Bulk alternative (recommended for contaminant detail):** ECHO SDWA download — https://echo.epa.gov/tools/data-downloads/sdwa-download-summary — ZIP of CSVs incl. `SDWA_PUB_WATER_SYSTEMS.csv`, violations, monitoring, keyed by `SUBMISSIONYEARQUARTER` + `PWSID`.
- **Commercial use:** Yes. **Cadence:** Quarterly (reporting lag). **Extract:** PWS identity/type, violations (MCL/MRDL exceedances `VIOL_MEASURE`), enforcement, service area.
- ⚠️ Underreporting/lag known; SDWIS_FED is thin on contaminant occurrence — use ECHO download + UCMR5 for actual results. PWSID = 2-letter state code + 7 digits.

2.2 EPA "Find Your Local CCR" + CCR Rule

- **CCR index:** https://www.epa.gov/ccr/ccr-information-consumers · **Find your CCR:** https://ofmpub.epa.gov/apex/safewater/f?p=136:102
- **There is NO centralized EPA CCR database/API.** CCRs are produced individually by **approximately 51,000 community water systems** (of the ~156,000 public water systems EPA monitors via SDWIS; per ODPHP Healthy People 2030: *"Of the 156,000 public water systems, approximately 51,000 are community water systems (CWS) that serve most people in the United States (a little less than 300 million)"*). They must be collected system-by-state.
- **CCR Rule revisions:** EPA finalized revisions requiring systems serving ≥10,000 to deliver reports **biannually**, with electronic/"plain-language" delivery phased in toward 2027.
- **Format:** PDF/HTML per utility. **Commercial use:** public documents, no aggregation license — treat as scrape.
- ⚠️ Even with the 2027 electronic-delivery push, there is still no single federal CCR feed — UCMR5 + SDWIS/ECHO remain the structured backbone.

2.3 EPA UCMR5 (PFAS + lithium)

- **Program:** https://www.epa.gov/dwucmr/fifth-unregulated-contaminant-monitoring-rule · **Data Finder:** https://www.epa.gov/dwucmr/fifth-unregulated-contaminant-monitoring-rule-data-finder
- **Occurrence text files:** https://www.epa.gov/dwucmr/occurrence-data-unregulated-contaminant-monitoring-rule — ZIP `ucmr5-occurrence-data.zip` with `UCMR5_All.txt`, `UCMR5_AddtlDataElem.txt`, `UCMR5_ZipCodes.txt`.
- **Format:** Excel `.xlsx` (Data Finder export, max 1M records) + tab-delimited TXT (bulk). ⚠️ **`UCMR5_All.txt` is too large for Excel** — use Access/DuckDB/pandas.

- **Auth:** None. **Commercial use:** Yes. **Cadence:** periodic. Per the EPA UCMR5 Data Finder: *"UCMR 5 data released to date include results received as of January 15, 2026, and represent approximately 95% of the total results that the EPA expects to receive. The final UCMR 5 data release will be in fall 2026."*
- **Extract:** PWSID, contaminant (29 PFAS + lithium), result value, MRL, ZIP codes served, sample location. Per EPA, **MRLs for the 29 PFAS range from 0.002 to 0.02 µg/L (2–20 ng/L); the lithium MRL is 9 µg/L.**
- ⚠️ UCMR5 results do NOT indicate MCL compliance; EPA intends to rescind/reconsider some PFAS determinations (PFHxS, PFNA, HFPO-DA/GenX, Hazard Index).

2.4 EPA Atrazine monitoring data

- No single dedicated atrazine API. Assemble from: **ECHO SDWA download tables** (§2.1) filtered to atrazine contaminant codes (CSV), plus the **Atrazine registration-review docket** EPA-HQ-OPP-2013-0266 on https://www.regulations.gov (PDF monitoring reports).
- **Format:** PDF (docket) + CSV (ECHO). **Commercial use:** Yes. ⚠️ No dedicated structured atrazine feed.

2.5 EWG Tap Water Database

- **URL:** https://www.ewg.org/tapwater/
- 🚫 **DO NOT USE.** EWG Terms of Use prohibit commercial use/reproduction of the database. Confirmed: **not an approved source for PurityIQ.** Use EPA primary sources (UCMR5, SDWIS/ECHO), which carry the same underlying data without licensing restrictions.

SECTION 3 — REGULATORY DIVERGENCE DATA (EU, UK, Japan, Canada)

3.1 EU Food Additives Database (Reg 1333/2008)

- **Commission DB:** https://food.ec.europa.eu/food-safety/food-improvement-agents/additives/database_en · **Legal text (Annexes II–V), EUR-Lex:** https://eur-lex.europa.eu/eli/reg/2008/1333/oj/eng (consolidated, Nov 2025)
- **Format:** Interactive HTML; **no reliable one-click CSV/Excel bulk export** of the full Union List. EUR-Lex Annex tables (HTML/PDF) are canonical.
- **Auth:** None. **Commercial use:** Yes (Commission reuse, Decision 2011/833/EU). **Attribution:** Required — "© European Union". ⚠️ No clean structured export confirmed on data.europa.eu — treat as HTML/PDF + LLM extraction.

3.2 EU Pesticide MRL Database

- **DB:** https://food.ec.europa.eu/plants/pesticides/eu-pesticides-database_en · **MRL download:** https://ec.europa.eu/food/plant/pesticides/eu-pesticides-database/start/screen/mrls/download · **data.europa.eu:** https://data.europa.eu/data/datasets/pesticides · **EURL Excel:** https://www.eurl-pesticides.eu
- **Format:** Online search + **Excel export**; portal states machine-to-machine **APIs available** under "Data download and APIs". **Auth:** None. **Commercial use:** Yes (© EU, attribution). **Cadence:** continuous.
- **Extract:** active substance × commodity MRL (mg/kg) with legislation links/history. ⚠️ Database "has no legal value" per disclaimer; Official Journal is authoritative.

3.3 UK Food Standards Agency — additives

- **Guidance/list:** https://www.food.gov.uk/business-guidance/approved-additives-and-e-numbers · **Official register (since Apr 1, 2025):** https://data.food.gov.uk/regulated-products (replaced statutory instruments as the GB Annex II/III list)
- **API — YES.** Docs: https://data.food.gov.uk/regulated-products/doc/reference ; base URI `http://data.food.gov.uk/regulated-products` ; returns **JSON**; supports additives, substances, groups, food categories with filtering. Catalogue: https://www.api.gov.uk/fsa/
- **Auth:** None. **Commercial use:** Yes — **Open Government Licence v3.0** (http://www.nationalarchives.gov.uk/doc/open-government-licence/version/3/). **Attribution:** Required (credit FSA + license link).
- **This is the cleanest EU/UK regulatory additive source — use as the structured backbone for divergence data.**

3.4 Japan MHLW / CAA food additives (English)

- **MHLW English:** https://www.mhlw.go.jp/stf/seisakunitsuite/bunya/kenkou_iryou/shokuhin/syokuten/index_00012.html · **CAA (jurisdiction moved Apr 1, 2024):** https://www.caa.go.jp/en/policy/standards_evaluation/food_additives_en
- English lists hosted by **JFCRF** (Japan Food Chemical Research Foundation), linked from official pages. Specifications & Standards 9th ed. (2022, English): https://dfa25.nihs.go.jp/jssfa/index.php (10th ed. 2024 is Japanese-only).
- **Format:** HTML lists + PDF compilations. No API/CSV. ⚠️ **Commercial use UNCLEAR:** official version is Japanese; English are courtesy translations; JFCRF is a foundation (not government) — verify terms before commercial redistribution; attribute source.

3.5 Canada — Health Canada Permitted Food Additives

- **Main:** https://www.canada.ca/en/health-canada/services/food-nutrition/food-safety/food-additives/lists-permitted.html (15 lists, legally incorporated by reference) · **Searchable viewer (not official):** https://health-infobase.canada.ca/nutrition/food-additives/ · **Open Gov catalogue:** https://open.canada.ca/data/en/dataset/51f47912-759c-4854-a50d-e38307200ed3
- **Format:** HTML tables; no CSV/API bulk export of official lists. **Auth:** None. **Commercial use:** Yes — **Open Government Licence – Canada** (attribution to Health Canada). **Extract:** permitted additive × food × max level.

3.6 California Prop 65 list

- **List page:** https://oehha.ca.gov/proposition-65/proposition-65-list
- **Format: Excel download** (includes listing mechanism, CAS#, listing date, and safe-harbor level where adopted) + PDF + WestLaw — the page's Excel export is the direct structured source.
- **NSRL/MADL safe-harbor levels (Excel export):** https://oehha.ca.gov/proposition-65/general-info/proposition-65-no-significant-risk-levels-nsrls-and-maximum-allowable-dose-levels-madls
- **Auth:** None. **Commercial use:** Yes (CA public data). **Cadence:** updated ≥ annually (often more frequently; the current list is the **December 5, 2025 Proposition 65 List**, with further 2026 listings in progress).
- **Extract:** chemical, CAS#, toxicity endpoint (cancer/reproductive), listing mechanism, listing date, safe-harbor level. Per OEHHA, the list *"has grown to include approximately 900 chemicals since it was first published in 1987."*

3.7 IARC Monographs

- **List of classifications:** https://monographs.iarc.who.int/agents-classified-by-the-iarc/ (section: https://monographs.iarc.who.int/list-of-classifications/)
- **Structured download — YES:** searchable/sortable spreadsheet (agent, group, CAS#, volume, year); legacy `List_of_Classifications.xls` + PDF exist; live HTML table is most current (Vols 1–141). Groups 1, 2A, 2B, 3.
- ⚠️ **Commercial use NOT clearly granted:** IARC is part of WHO; default licensing is often **CC BY-NC-SA 3.0 IGO (non-commercial)**. Verify / seek permission (imo@iarc.who.int) before commercial use. Attribution to IARC required.

SECTION 4 — BARCODE & PRODUCT DATA

4.1 Open Food Facts API

- **Docs:** https://openfoodfacts.github.io/openfoodfacts-server/api/ · v2 reference https://openfoodfacts.github.io/openfoodfacts-server/api/ref-v2/

- **Get product by barcode:** `GET https://world.openfoodfacts.org/api/v2/product/{barcode}.json` — field filter `?fields=product_name,brands,ingredients_text,ingredients,categories`
- **Auth:** None for reads (writes use username/password; migrating to Keycloak/OAuth2). **Rate limits:** 100 req/min product reads, 10 req/min search/facet; HTTP 503 if exceeded. **1 API call = 1 real scan**; for bulk use nightly exports (MongoDB dump, JSONL, Parquet) — do not scrape.
- **License/attribution:** Database under **ODbL**; contents under Database Contents License; images CC-BY-SA. **Commercial use: Yes.** Must attribute "Open Food Facts" + ODbL and share-alike adapted databases.
- **Extract:** `product_name` , `brands` , `ingredients_text` , `ingredients` (array), `categories` , `additives_tags` , `allergens` .
- **Python:**

```
import requests
bc = "3017624010701"
r = requests.get(f"https://world.openfoodfacts.org/api/v2/product/{bc}.json",
    params={"fields":"product_name,brands,ingredients_text,ingredients,categories"},
    headers={"User-Agent":"PurityIQ/1.0 (contact@purityiq.app)"}, timeout=15)
data = r.json()
product = data.get("product") if data.get("status") == 1 else None
```

- **Node.js:**

```
const bc = "3017624010701";
const url = `https://world.openfoodfacts.org/api/v2/product/${bc}.json?fields=product_name,b
const res = await fetch(url, { headers: { "User-Agent": "PurityIQ/1.0 (contact@purityiq.app)
const data = await res.json();
const product = data.status === 1 ? data.product : null;
```

- ⚠️ Always send a descriptive `User-Agent` . Staging host `world.openfoodfacts.net` requires basic auth and is test-only.

4.2 Open Beauty Facts API

- **Site:** https://world.openbeautyfacts.org/ · **Data:** https://world.openbeautyfacts.org/data
- **Same structure as OFF** (98% identical): Openfoodfacts `GET https://world.openbeautyfacts.org/api/v2/product/{barcode}.json` . Nutri-Score/Eco-Score/Knowledge Panels NOT supported. Openfoodfacts
- **License:** ODbL, commercial use permitted, attribution required. Open Beauty Facts
- ⚠️ **WEAK US COSMETICS COVERAGE:** global DB on the order of **50,000+ products**, Europe-skewed, sparse/missing structured ingredient lists, few product images. Inadequate as a sole source for the US cosmetics/skincare market.

4.3 Alternative cosmetic barcode databases (US coverage)

- **California Safe Cosmetics "Chemicals in Cosmetics" (§6.2)** — best *free, commercially-licensed* US-specific structured ingredient/hazard dataset (CC-BY).
- **The Beauty API** (https://thebeautyapi.com) — commercial, ~180,000 products, INCI-mapped ingredients, comedogenicity/irritancy ratings, 18GB image archive, Thebeautyapi JSONL/CSV. Paid; best day-one US coverage. (Vendor marketing claims — verify pricing/terms.)
- **UPCitemdb** (https://www.upcitemdb.com/api/explanation) and **Barcode Lookup API** (https://www.barcodelookup.com/api) — broad UPC→name/brand coverage incl. US cosmetics; commercial tiers; good for resolution, light on ingredients.
- **Spoonacular** — food-focused; does NOT cover cosmetics. **CosDNA / INCI Decoder** — useful references but no official commercial API / scraping restricted; not recommended. **Sephora** — no public commercial API.
- **Recommendation:** Open Beauty Facts (free, ODbL) + California Safe Cosmetics (free, CC-BY) as the open baseline; layer a paid commercial DB (The Beauty API or Barcode Lookup) for US barcode→product resolution and images.

4.4 Barcode scanning SDK for iOS (Swift)

- **Recommended: Apple VisionKit `DataScannerViewController`** (iOS 16+, built-in, Phunware free, no third-party dependency). Fallback: **Vision `VNDetectBarcodesRequest` + AVFoundation** for custom UI / iOS <16.
- **Requires:** `NSCameraUsageDescription` in Info.plist; `DataScannerViewController.isSupported && .isAvailable` Medium (needs A12 Bionic+). Phunware
- **Swift (VisionKit):**

```
import VisionKit

let scanner = DataScannerViewController(
    recognizedDataTypes: [.barcode(symbologies: [.ean13, .ean8, .upce, .code128])],
    qualityLevel: .balanced,
    recognizesMultipleItems: false,
    isHighlightingEnabled: true)

// DataScannerViewControllerDelegate:
func dataScanner(_ s: DataScannerViewController, didAdd added: [RecognizedItem], allItems:
    for item in added {
        if case let .barcode(bc) = item { print(bc.payloadStringValue ?? "") }
    }
}
try? scanner.startScanning()
```

- **Commercial use:** Yes (first-party Apple framework, no fees). Request **EAN-13/UPC-A** for retail products.

SECTION 5 — AI PDF PIPELINE (DEEPSEEK + INFRASTRUCTURE)

5.1 DeepSeek API

- **Base URL (OpenAI-compatible):** `https://api.deepseek.com` (chat at `/chat/completions`). Anthropic-compatible: `https://api.deepseek.com/anthropic`.
- **Docs:** https://api-docs.deepseek.com · **Pricing:** https://api-docs.deepseek.com/quick_start/pricing · **Signup:** https://platform.deepseek.com (5M free tokens on registration, no credit card).
- **Auth:** `Authorization: Bearer $DEEPSEEK_API_KEY`.
- **Current model IDs (2026):** `deepseek-v4-flash` (cost-efficient), `deepseek-v4-pro` (flagship reasoning). **Legacy `deepseek-chat` / `deepseek-reasoner` retire July 24, 2026, 15:59 UTC** (both map to V4-Flash modes today) — use explicit IDs.
- **Pricing (per 1M, direct):** V4 Flash $0.14 input Requesty / $0.0028 cached / $0.28 output. Requesty V4 Pro $1.74 / $0.0145 cached / $3.48 output (steady-state after 75% promo ending May 31, 2026). 1M context, Requesty 384K max output. Context caching default-on (~98% cheaper hits).
- **DeepSeek-OCR: NOT on the managed api.deepseek.com.** Open-weights, MIT-licensed, **self-host only** — DeepSeek-OCR: https://huggingface.co/deepseek-ai/DeepSeek-OCR · DeepSeek-OCR-2 (Jan 2026): https://huggingface.co/deepseek-ai/DeepSeek-OCR-2 · GitHub: https://github.com/deepseek-ai/DeepSeek-OCR. Run via vLLM/Transformers on your own GPU. Not confirmed on Together/Fireworks serverless menus as of June 2026.
- ⚠️ **Data governance:** DeepSeek's direct API is hosted in China Nxcode — for US data-governance, route through a US provider (§5.4).

5.2 Together AI

- **Base URL:** `https://api.together.xyz/v1` (also `https://api.together.ai`); chat `/chat/completions`. **Signup:** https://www.together.ai (+$5 free credits; min $5 purchase; Startup Accelerator credits available).
- **Auth:** `Authorization: Bearer $TOGETHER_API_KEY`.
- **DeepSeek model IDs:** `deepseek-ai/DeepSeek-V4-Pro`, `deepseek-ai/DeepSeek-V3.1`, `deepseek-ai/DeepSeek-R1`, `deepseek-ai/DeepSeek-V3`.
- **Pricing (per 1M):** V4 Pro $2.10/$4.40; V3.1 $0.60/$1.70; R1 $3.00/$7.00. Flat per-token (no cached-input discount as of June 6, 2026).

- **curl:**

```
curl -X POST "https://api.together.xyz/v1/chat/completions" \
  -H "Authorization: Bearer $TOGETHER_API_KEY" -H "Content-Type: application/json" \
  -d '{"model":"deepseek-ai/DeepSeek-V4-Pro","messages":[{"role":"user","content":"Extract c
```

5.3 Fireworks AI

- **Base URL:** `https://api.fireworks.ai/inference/v1` (OpenAI-compatible); chat `/chat/completions`. **Docs:** https://docs.fireworks.ai · **Pricing:** https://docs.fireworks.ai/serverless/pricing · **DeepSeek hub:** https://fireworks.ai/deepseek · **Signup:** https://fireworks.ai ($1 free credits; postpaid pay-per-token).
- **Auth:** `Authorization: Bearer $FIREWORKS_API_KEY`.
- **Model ID format:** `accounts/fireworks/models/{model}` — e.g., `accounts/fireworks/models/deepseek-v3p2`, TypingMind `accounts/fireworks/models/deepseek-r1`. DeepSeek V4 is live Fireworks AI ("single click") — confirm exact V4 slug in the model library at deploy time.
- **Pricing:** cached input default 50% of input; batch 50%; e.g., DeepSeek V3.2 $0.56 input / $1.68 output per 1M. Day-0 support for new DeepSeek releases. Fireworks AI
- **Python (OpenAI SDK):**

```
import os
from openai import OpenAI
client = OpenAI(api_key=os.environ["FIREWORKS_API_KEY"],
                base_url="https://api.fireworks.ai/inference/v1")
resp = client.chat.completions.create(
    model="accounts/fireworks/models/deepseek-v3p2",
    messages=[{"role":"user","content":"Extract contaminant results as JSON."}])
print(resp.choices[0].message.content)
```

5.4 Recommendation: Fireworks vs Together

Use Fireworks AI for the DeepSeek PDF pipeline; keep Together as failover. Reasoning:

1. **US data governance** — both Fireworks and Together are US-hosted (vs DeepSeek-direct in China), satisfying the "keep infra in the US" requirement. Together states API data is not used for training; Fireworks runs production-grade US infra.
2. **Cost/throughput** — Fireworks' FireAttention engine + on-demand tier (~250% throughput, ~50% lower latency than vLLM) Fireworks AI plus default **50% cached-input + 50% batch discounts** materially cut cost for the high-volume, repetitive-prompt PDF-extraction workload (same system prompt every doc → big cache wins). Together is flat-rate with no cached-input discount.
3. **Day-0 model availability** — Fireworks shipped DeepSeek V4 on launch day. Fireworks AI

4. **Tie-breaker** — if you later self-host **DeepSeek-OCR** for scanned PDFs, Fireworks' on-demand GPUs keep it one-vendor. Both are OpenAI-compatible, so swapping is a base-URL + model-ID change.

5.5 PDF text extraction (pre-LLM)

- **Default: PyMuPDF (`fitz`) —** fastest, best text+layout, free/local. Use **pdfplumber** for table-cell geometry (CCR/USDA tables). Reserve **AWS Textract / Google Document AI** (paid, per-page) only for **scanned/image PDFs** with no text layer (then optionally self-hosted DeepSeek-OCR as a free alternative).
- **Cost-effectiveness:** PyMuPDF/pdfplumber are free and local → most cost-effective for the bulk of government PDFs (which have text layers). Textract (~$1.50/1,000 pages) only for scans.
- **Python (PyMuPDF):**

```
import fitz
doc = fitz.open("ccr.pdf")
text = "\n".join(page.get_text() for page in doc)  # send `text` to the LLM
```

- **Python (pdfplumber, tables):**

```
import pdfplumber
with pdfplumber.open("usda.pdf") as pdf:
    tables = [t for pg in pdf.pages for t in pg.extract_tables()]
```

- **Node.js:**

```
import fs from "node:fs";
import pdf from "pdf-parse";
const data = await pdf(fs.readFileSync("ccr.pdf"));
console.log(data.text);
```

- **Pipeline:** extract text locally → chunk → DeepSeek V4 Flash (Fireworks) with JSON-schema prompt → validate → Firestore. Fall back to OCR only for pages where extracted text is empty.

5.6 Firebase Cloud Functions v2

- **Storage trigger import (Node.js):** `const {onObjectFinalized} = require("firebase-functions/v2/storage");` Firebase
- **Storage trigger import (Python):** `from firebase_functions import storage_fn` Firebase
- **Node.js:**

```
const {onObjectFinalized} = require("firebase-functions/v2/storage");
const {initializeApp} = require("firebase-admin/app");
const {getStorage} = require("firebase-admin/storage");
```

```
initializeApp();

exports.processPdf = onObjectFinalized(
  { bucket: "purityiq-docs", timeoutSeconds: 300, memory: "1GiB" },
  async (event) => {
    const filePath = event.data.name;
    if (event.data.contentType !== "application/pdf") return null;
    // download, extract, call LLM, write Firestore
  });
```

- **Python:**

```
from firebase_functions import storage_fn, options
from firebase_admin import initialize_app
initialize_app()

@storage_fn.on_object_finalized(bucket="purityiq-docs", timeout_sec=300,
                                memory=options.MemoryOption.GB_1)
def process_pdf(event: storage_fn.CloudEvent[storage_fn.StorageObjectData]):
    if event.data.content_type != "application/pdf":
        return
    file_path = event.data.name
    # download, extract, call LLM, write Firestore
```

- ⚠️ **Known issues:** v2 requires the **Blaze (pay-as-you-go) plan.** Firebase v2 storage triggers **require a bucket filter** (firebase-tools errors "storage event trigger is missing bucket filter" if omitted). Cannot upgrade a v1→v2 function with the same name. Firebase Folder creation in the emulator can spuriously fire `onObjectFinalized`.

SECTION 6 — COSMETICS REGULATORY DATA (V2)

6.1 EU Cosmetics Regulation 1223/2009 (Annexes II–VI)

- **CosIng (current 2026 URL, post-migration):** https://single-market-economy.ec.europa.eu/sectors/cosmetics/cosmetic-ingredient-database_en (glossary: …/cosmetic-ingredient-database/cosing-glossary-ingredients_en). Legacy https://ec.europa.eu/growth/tools-databases/cosing/ redirects.
- **Legal text (EUR-Lex):** https://eur-lex.europa.eu/eli/reg/2009/1223/oj/eng
- **Annexes:** II (prohibited), III (restricted), IV (colorants), V (preservatives), VI (UV filters).
- **Format:** Searchable HTML; **post-migration bulk export is unreliable/limited** — use EUR-Lex Annex tables (HTML/PDF) as canonical machine-usable source + LLM extraction.
- **Auth:** None. **Commercial use:** Yes (© EU, attribution). ⚠️ Verify the CosIng export button at integration time.

6.2 California Safe Cosmetics Program (“Chemicals in Cosmetics”)

- **Dataset (CA Open Data):** https://data.ca.gov/dataset/chemicals-in-cosmetics · **Mirror:** https://data.chhs.ca.gov/dataset/chemicals-in-cosmetics
- **DIRECT CSV:** https://data.chhs.ca.gov/dataset/596b5eed-31de-4fd8-a645-249f3f9b19c4/resource/57da6c9a-41a7-44b0-ab8d-815ff2cd5913/download/cscpopendata.csv
- **Public search UI:** https://cscpsearch.cdph.ca.gov/search/publicsearch
- **Format:** CSV (main) + TSV/JSON/XML + CKAN DataStore API; subcategories XLSX; PDF data dictionary. **Structured download + API: YES.**
- **Auth:** None. **Commercial use:** Yes — **CC-BY** (canonical data.ca.gov listing). **Attribution:** “California Safe Cosmetics Program, California Department of Public Health.” CA **Cadence:** daily; coverage 2007–present. CA
- **Extract:** product label name, company/brand, category, CAS#, chemical name, # chemicals/product, reporting/discontinuation/reformulation dates. **Best free, commercially-licensed US cosmetics ingredient/hazard dataset.**

6.3 FDA Prohibited & Restricted Ingredients in Cosmetics

- **URL:** https://www.fda.gov/cosmetics/cosmetics-laws-regulations/prohibited-restricted-ingredients-cosmetics
- **Format: HTML-only narrative** (~11 prohibited/restricted categories). Legal source: 21 CFR 700.11–700.35 (eCFR, HTML/PDF). No CSV/API.
- **Auth:** None. **Commercial use:** Yes (US public domain). **Extract:** prohibited/restricted ingredients (bithionol, chloroform, mercury compounds, methylene chloride, vinyl chloride, hexachlorophene, CFC propellants, halogenated salicylanilides, zirconium aerosol complexes, etc.).

6.4 Open Beauty Facts — US coverage assessment

- See §4.2. **US coverage is weak** (global DB ~50,000+ products, Europe-skewed, sparse ingredients, few images). Use only as a free supplement; pair with California Safe Cosmetics + a paid commercial DB for US production coverage.

SECTION 7 — INFRASTRUCTURE

7.1 Firebase Firestore (Admin SDK)

- **Node.js:** `npm i firebase-admin firebase-functions` · **Python:** `pip install firebase-admin firebase-functions`

- **Node.js init + write (inside a Cloud Function):**

```
const {initializeApp} = require("firebase-admin/app");
const {getFirestore} = require("firebase-admin/firestore");
initializeApp();
const db = getFirestore();
await db.collection("contaminants").doc(pwsid).set(record, { merge: true });
```

- **Python init + write:**

```
from firebase_admin import initialize_app, firestore
initialize_app()
db = firestore.client()
db.collection("contaminants").document(pwsid).set(record, merge=True)
```

7.2 GitHub Actions — weekly USDA PDP check → Firebase Storage

```
name: weekly-pdp-fetch
on:
  schedule:
    - cron: "0 8 * * 1"   # Mondays 08:00 UTC
  workflow_dispatch:
jobs:
  fetch:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install requests google-cloud-storage
      - name: Fetch latest PDP and upload
        env:
          GCP_SA_KEY: ${{ secrets.GCP_SA_KEY }}
        run: |
          echo "$GCP_SA_KEY" > /tmp/sa.json
          export GOOGLE_APPLICATION_CREDENTIALS=/tmp/sa.json
          python scripts/fetch_pdp.py
```

```
# scripts/fetch_pdp.py
import requests, datetime
from google.cloud import storage
YEAR = datetime.date.today().year - 1
url = f"https://www.ams.usda.gov/sites/default/files/media/{YEAR}PDPDatabase.zip"
r = requests.get(url, timeout=300)
if r.status_code == 200:
    storage.Client().bucket("purityiq-docs").blob(f"pdp/{YEAR}.zip").upload_from_string(r.co
```

- **Secret:** `GCP_SA_KEY` = JSON service-account key with Storage Object Admin on the bucket. ⚠️

PDP filenames have historically drifted — try the documented pattern, then fall back to scraping the landing page for the current ZIP link.

7.3 Together / Fireworks API key provisioning

- **Fireworks:** https://fireworks.ai → Dashboard → API Keys → "Create API Key" → store as `FIREWORKS_API_KEY` (env var / GitHub secret / `firebase functions:secrets:set`). $1 free credits; add billing for production.
- **Together:** https://api.together.ai → Settings → API Keys → generate → store as `TOGETHER_API_KEY`. $5 free credits; min $5 purchase.
- **DeepSeek (optional fallback):** https://platform.deepseek.com → API Keys → generate → `DEEPSEEK_API_KEY`.
- **Never embed keys in the iOS client** — all LLM calls go through Firebase Functions (server-side proxy).

KNOWN ISSUES / WATCH OUT FOR (consolidated)

- **USDA PDP:** filename drift; 2-digit year-token column names; varying detection limits/commodities by year.
- **AB 899:** no central portal — per-manufacturer scraping, inconsistent formats, some gated by CAPTCHA/UPC.
- **eCFR / EU additives / EU cosmetics / FDA pages:** XML or HTML/PDF only — require LLM extraction, no clean CSV.
- **UCMR5 `UCMR5_All.txt`:** too large for Excel; use DuckDB/Access/pandas. MRLs 0.002–0.02 µg/L (PFAS), 9 µg/L (lithium).
- **CCRs:** no federal aggregated feed; ~51,000 utility PDFs; CCR Rule revisions toward 2027 may add electronic delivery.
- **DeepSeek direct API:** China-hosted (data governance) + legacy aliases retire July 24, 2026.
- **DeepSeek-OCR:** self-host only (MIT), not on managed APIs.
- **Firebase v2:** Blaze plan required; storage triggers need bucket filter; no same-name v1→v2 upgrade.
- **IARC Monographs:** likely WHO non-commercial license — verify before commercial use.
- **EWG Tap Water DB:** 🚫 commercial use prohibited — excluded.
- **Open Beauty Facts:** weak US cosmetics coverage; ODbL share-alike obligations on adapted databases.

RECOMMENDATIONS (staged)

1. **Phase 1 (MVP):** VisionKit scanning → Open Food Facts API (food) + California Safe Cosmetics CSV (cosmetics) → Firestore. Add eCFR + EPA UCMR5 + openFDA enforcement as the structured "hazard" reference tables. All free, all commercially licensed.
2. **Phase 2 (PDF pipeline):** Firebase Functions v2 `onObjectFinalized` → PyMuPDF extraction → DeepSeek V4 Flash on **Fireworks AI** → JSON schema → Firestore. Add GitHub Actions weekly jobs for USDA PDP, UCMR5, and the Prop 65 Excel export.
3. **Phase 3 (coverage + scale):** Layer a paid commercial cosmetics DB (Barcode Lookup / The Beauty API) for US barcode→product resolution + images; add EU/UK/Japan/Canada divergence tables (start with UK FSA JSON API — the cleanest). Escalate to DeepSeek V4 Pro only where Flash extraction quality is insufficient.

- **Thresholds that change the plan:** if PDF volume × cache-hit rate pushes Fireworks serverless cost above self-hosting break-even, deploy DeepSeek-OCR/V4 on Fireworks on-demand GPUs (or your own). If Open Beauty Facts US scan hit-rate stays <50%, prioritize the paid cosmetics DB sooner. If DeepSeek's direct API ever becomes a data-governance blocker mid-build, the OpenAI-compatible design means swapping to Fireworks/Together is a base-URL + model-ID change.

CAVEATS

- Pricing and model IDs (DeepSeek V4, Together, Fireworks) are fast-moving 2026 figures drawn partly from vendor/marketing pages; re-verify at integration. DeepSeek legacy aliases hard-retire July 24, 2026, 15:59 UTC.
- Several regulatory "databases" are legally non-authoritative (EU pesticide DB disclaimer; Canada searchable viewer; EU additives DB) — the legal text (EUR-Lex, CFR, Official Journal) governs.
- IARC and Japan/JFCRF commercial licensing is unconfirmed — obtain written permission before shipping their data commercially.
- The exact Fireworks DeepSeek V4 model slug should be confirmed in the live model library at deploy time (V3.2 slug `accounts/fireworks/models/deepseek-v3p2` is verified; V4 was live as of the DeepSeek hub but the precise slug should be read from the console).